

Design of a RISC-V Microprocessor

Course: 02114

Group: 10

Students: August Dyrlov Madsen (s234357) and Niels Laurits T. Meldgaard (s230482)

Repository: https://github.com/NielsLauritsMeldgaard/RISC_V_GRP_10

Abstract

This project presents the design, implementation, and FPGA validation of a compact 32-bit RISC-V microprocessor on the Digilent Basys3 board. The processor targets the RV32I base ISA and is implemented in SystemVerilog as a simple, three-stage pipeline inspired by Schoeberl's Wildcat organization¹. The pipeline is partitioned into instruction fetch (IF), instruction decode (ID), and execute (EX), with a single-cycle write-back path from EX to the register file, a small forwarding unit to mitigate data hazards, and a flush policy to ensure correct control-flow behavior on taken branches and jump instructions. The register file is implemented using distributed FPGA resources.

To interface memory and peripherals in a scalable way, the design employs a Wishbone-style interconnect with the CPU acting as bus master that engages with its slaves following a simple handshake protocol², making it easier to potentially transitioning into using slow but big external memory. Instruction fetch uses a dedicated instruction bus, while loads/stores and memory-mapped I/O (MMIO) use a separate data bus. Both buses operate within a shared 32-bit physical address space, where the most significant address bits select between boot ROM, instruction RAM, data RAM, and peripheral registers. The MMIO subsystem exposes Basys3 I/O devices, including LEDs/switches, seven-segment display, UART status/data, and button inputs; a PS/2 interface is included as a placeholder for future extension.

A small boot ROM at address 0x00000000 contains a UART bootloader (written in RISC-V assembly) that receives a program image over serial and writes it into on-chip instruction BRAM. When a magic termination word (0xDEADBEEF) is received, execution transfers to the beginning of instruction RAM (0x10000000). This mechanism enables a rapid software/hardware development loop, as programs compiled to raw binaries from C can be uploaded and executed without rerunning the FPGA synthesis and implementation. the bootloader program is very small (32words).

Functional verification is performed through an automated simulation-based test flow using Vivado xsim, comparing architectural state (e.g., register contents) against expected results. Continuous integration is used primarily to detect fatal build/synthesis issues. Overall, the final system demonstrates a fully working RV32I-based microprocessor capable of bare-metal applications on FPGA, while maintaining a design structure that supports future extensions such as external memory integration via the existing bus and stalling mechanism.

Address map overview (shared 32-bit address space)

Table 1: Shared 32-bit address map for instruction and data access.

Region	Base	Bus	Function	Notes
Boot ROM	0x0000.0000	I-bus	Reset/boot code	UART bootloader
Instruction RAM	0x1000.0000	I-bus	Instruction fetch	Program execution
Instruction RAM	0x1000.0000	D-bus	Program upload writes	Used by bootloader
Data RAM	0x2000.0000	D-bus	Loads/stores	Synchronous BRAM
MMIO (I/O)	0x4000.0000	D-bus	Peripherals	See offsets below
MMIO offset	Address	Description		
0x0	0x4000.0000	LEDs write / switches+LEDs readback		
0x4	0x4000.0004	Seven-segment register		
0x8	0x4000.0008	UART status/data (rx.valid, tx-busy, rx.data)		
0xC	0x4000.000C	Buttons / PS2 status (PS2 optional)		

¹<https://github.com/schoeberl/wildcat>

²a strobe signal is sent from the master along with address and data, if an acknowledge signal by the slave is sent in same cycle, the design is set up to potentially stall the CPU until the data is ready.

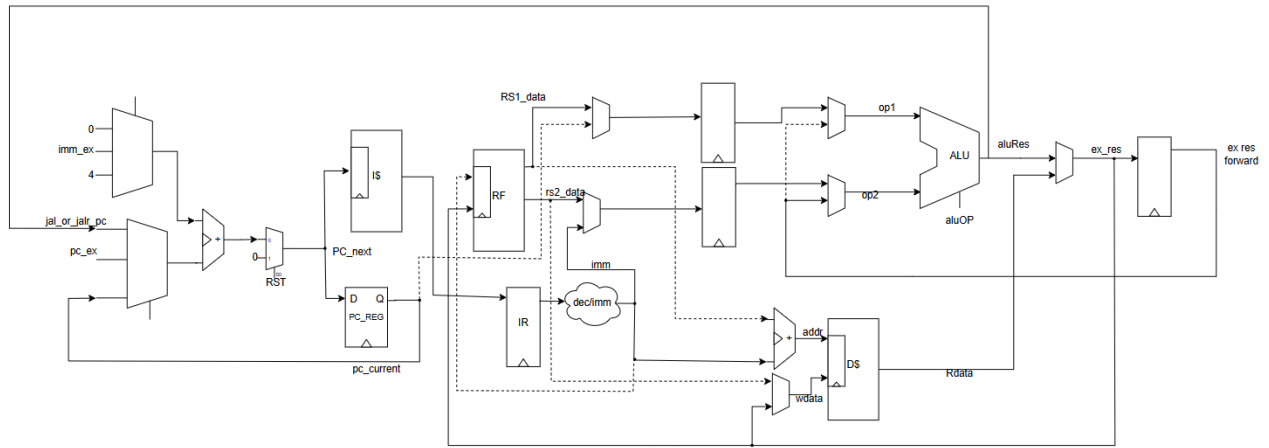


Figure 1: Simplified block diagram of the implemented CPU

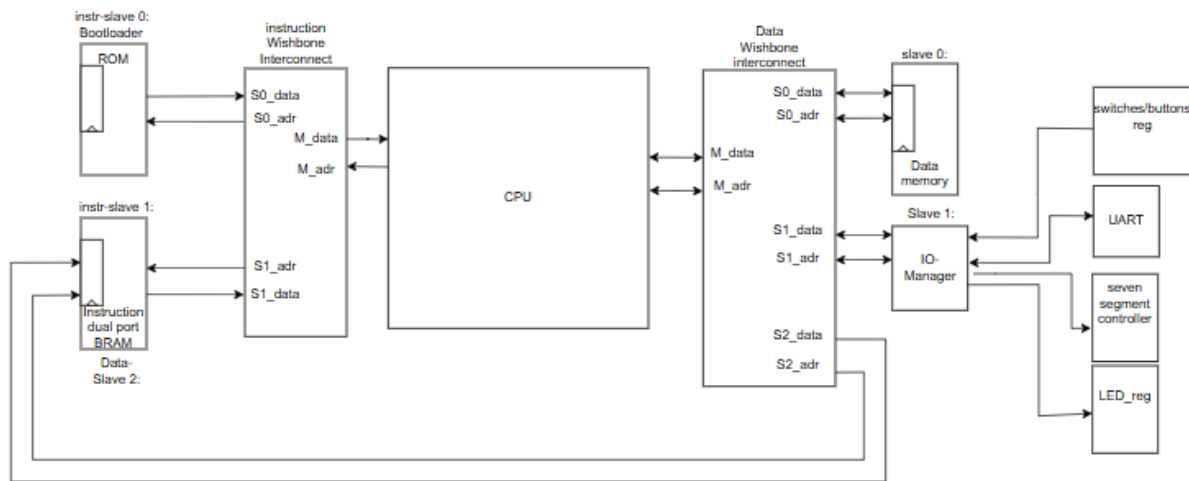


Figure 2: CPU and Bus system with all memories and MMIO.

Resources and performance

The final processor can run 100MHz if the memory is not too big, and the resource usage is as follows:

Table 2: FPGA Resource Utilization Summary

Name	Slice LUTs	Slice Registers	F7 Muxes	Slices	LUT as Logic
datapath	1643	583	63	519	1599
ex_stage (EX)	732	193	63	264	732
ALU_unit (ALU)	270	0	32	125	270
id_stage (ID)	328	64	0	176	284
regFile (regFile)	267	0	0	119	223
peripherals (io_manager)	503	290	0	215	503
button_unit (debounce)	196	104	0	64	196
seg_unit (seven_seg_controller)	200	48	0	80	200
uart_unit (uart_controller)	107	73	0	37	107